

```

"""
Thin wrapper around The Odds API (https://the-odds-api.com).

Docs: https://the-odds-api.com/liveapi/guides/v4/
Free tier: 500 requests/month. /v4/sports does NOT count against quota.
"""

import requests

BASE_URL = "https://api.the-odds-api.com/v4"

class OddsAPIError(Exception):
    pass

class OddsClient:
    def __init__(self, api_key: str):
        self.api_key = api_key

    def _get(self, path: str, params: dict | None = None) -> dict | list:
        params = dict(params or {})
        params["apiKey"] = self.api_key
        resp = requests.get(f"{BASE_URL}{path}", params=params, timeout=15)
        if resp.status_code != 200:
            raise OddsAPIError(f"{resp.status_code}: {resp.text[:300]}")
        return resp.json()

    def list_sports(self, all_sports: bool = False) -> list[dict]:
        """Free call. Returns sport_key, title, group, active, etc."""
        params = {"all": "true"} if all_sports else {}
        return self._get("/sports", params)

    def get_odds(
        self,
        sport_key: str,
        regions: str = "us,uk,eu",
        markets: str = "h2h",
        odds_format: str = "decimal",
    ) -> list[dict]:
        """
        Returns live odds for all upcoming events in a sport.
        Each event has a list of bookmakers, each with markets/outcomes.
        """

```

```

        params = {
            "regions": regions,
            "markets": markets,
            "oddsFormat": odds_format,
        }
        return self._get(f"/sports/{sport_key}/odds", params)

def get_event_odds(
    self,
    sport_key: str,
    event_id: str,
    regions: str = "us,uk,eu",
    markets: str = "h2h",
    odds_format: str = "decimal",
) -> dict:
    params = {
        "regions": regions,
        "markets": markets,
        "oddsFormat": odds_format,
    }
    return self._get(f"/sports/{sport_key}/events/{event_id}/odds", params)

def best_prices(self, event: dict, market_key: str = "h2h") -> dict:
    """
    Given one event dict from get_odds(), return the best available
    price per outcome across all bookmakers.
    {outcome_name: {"price": float, "bookmaker": str}}
    """
    best: dict[str, dict] = {}
    for bm in event.get("bookmakers", []):
        for market in bm.get("markets", []):
            if market.get("key") != market_key:
                continue
            for outcome in market.get("outcomes", []):
                name = outcome["name"]
                price = outcome["price"]
                if name not in best or price > best[name]["price"]:
                    best[name] = {"price": price, "bookmaker": bm["title"]}
    return best

```